

Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Experiment 1

Implementation of a Single Layer Perceptron for Binary Classification

1. Objective

The objective of this experiment is to understand the concept of an artificial neuron and implement a Single Layer Perceptron from scratch for binary classification. Students will learn the perceptron learning algorithm, understand the importance of activation functions, visualize the learning process, and evaluate the classifier using a real-world dataset.

2. Background Theory

An Artificial Neuron is the fundamental building block of a neural network. It receives one or more input features, multiplies them by corresponding weights, adds a bias term, and applies an activation function to determine the output.

The Single Layer Perceptron is the earliest supervised learning algorithm capable of solving linearly separable binary classification problems.

Mathematical Formulation

Weighted Sum

$$z = \mathbf{w}^T \mathbf{x} + b$$

where

- $\mathbf{x}$: Input vector
- $\mathbf{w}$: Weight vector
- b : Bias
- z : Linear combination

Prediction

$$\hat{y} = f(z)$$

Weight Update Rule

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \eta(y - \hat{y})\mathbf{x}$$

Bias Update

$$b_{new} = b_{old} + \eta(y - \hat{y})$$

where η denotes the learning rate.

3. Activation Functions

An activation function determines the output of a neuron based on the weighted sum of its inputs. It introduces non-linearity, enabling neural networks to learn complex patterns. Although the perceptron uses only the Step Activation Function, modern deep learning models employ differentiable activation functions for efficient optimization through backpropagation.

Step Activation Function

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

The Step Activation Function produces a binary output and is therefore suitable only for linearly separable binary classification problems.

Activation	Output Range	Typical Applications
Step	$\{0, 1\}$	Classical Perceptron
Sigmoid	$(0, 1)$	Binary Classification Output Layer
Tanh	$(-1, 1)$	Hidden Layers (Traditional Networks)
ReLU	$[0, \infty)$	Hidden Layers in CNNs and MLPs
Leaky ReLU	$(-\infty, \infty)$	Avoiding Dying ReLU
Softmax	$(0, 1)$	Multi-class Classification Output Layer

Note: This experiment uses only the Step Activation Function. The remaining activation functions will be studied in subsequent experiments involving Multilayer Perceptrons and Deep Neural Networks.

4. Dataset Description

Dataset: Banknote Authentication Dataset

Source: UCI Machine Learning Repository

Download Link:

<https://archive.ics.uci.edu/dataset/267/banknote+authentication>

Instances	1372
Features	4 Numerical Features
Classes	2
Missing Values	None
Task	Binary Classification

Feature Description

- Variance
- Skewness
- Curtosis
- Entropy

Target Class

- 0 – Authentic Banknote
- 1 – Forged Banknote

5. Experimental Procedure

Task 1: Dataset Exploration

- Load the dataset.
- Display the first five samples.
- Determine dataset dimensions.
- Identify missing values.
- Display descriptive statistics.

Expected Output

Dataset summary and statistical information.

Task 2: Exploratory Data Analysis

Generate

- Histograms
- Correlation Heatmap
- Scatter Plot
- Boxplots

Task 3: Data Preprocessing

- Normalize all numerical features.
- Split the dataset into Training (80%) and Testing (20%).

Task 4: Perceptron Implementation

Implement from scratch

- Weight Initialization
- Bias Initialization
- Step Activation Function
- Forward Propagation
- Perceptron Learning Rule

Task 5: Model Training

Display

- Epoch Number
- Misclassified Samples
- Updated Weights
- Updated Bias

Task 6: Model Evaluation

Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

6. Mandatory Plots

Plot	Purpose	Expected Inference
Histogram_Variance.eps Histogram_Skewness.eps Histogram_Curtosis.eps Histogram_Entropy.eps	Distribution Analysis	Identify skewness and outliers.

Boxplot_Variance.eps Boxplot_Skewness.eps Boxplot_Curtosis.eps Boxplot_Entropy.eps	Distribution Analysis	Compare quartiles, medians, and outliers.
Correlation_Heatmap.eps	Feature Relationship	Observe highly correlated features.
Scatter_Plot.eps	Class Distribution	Determine whether classes are approximately linearly separable.
Training_Error_vs_Epoch.eps	Learning Behaviour	Verify convergence of the perceptron.
Weight_Evolution.eps	Parameter Analysis	Observe stabilization of learned weights.
Bias_Evolution.eps	Parameter Analysis	Understand the role of the bias term.
Confusion_Matrix.eps	Performance Evaluation	Interpret TP, FP, TN and FN.
Learning_Rate_Comparison.eps	Hyperparameter Study	Compare convergence behaviour for different learning rates.

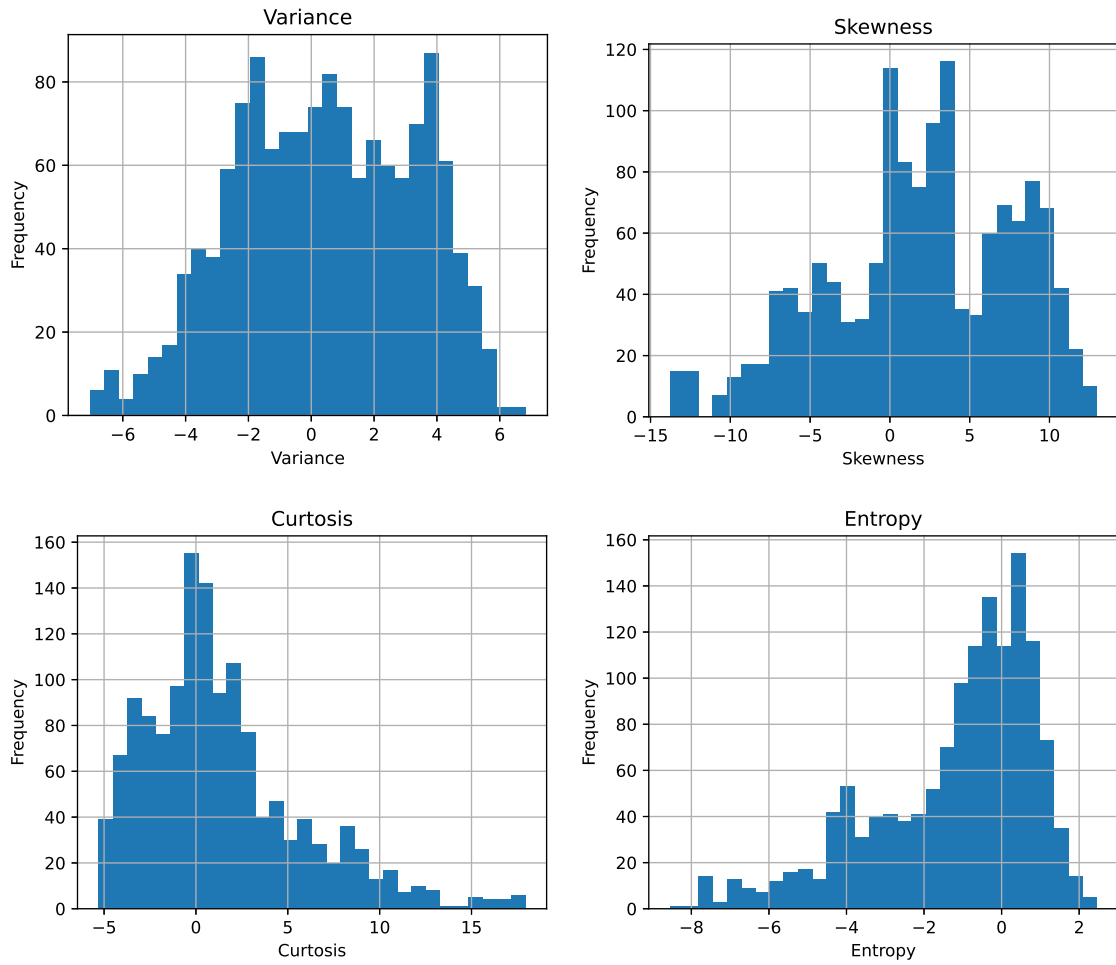


Figure 1: The histograms show that variance and skewness have clear separation between authentic and forged notes, while kurtosis and entropy overlap a lot.

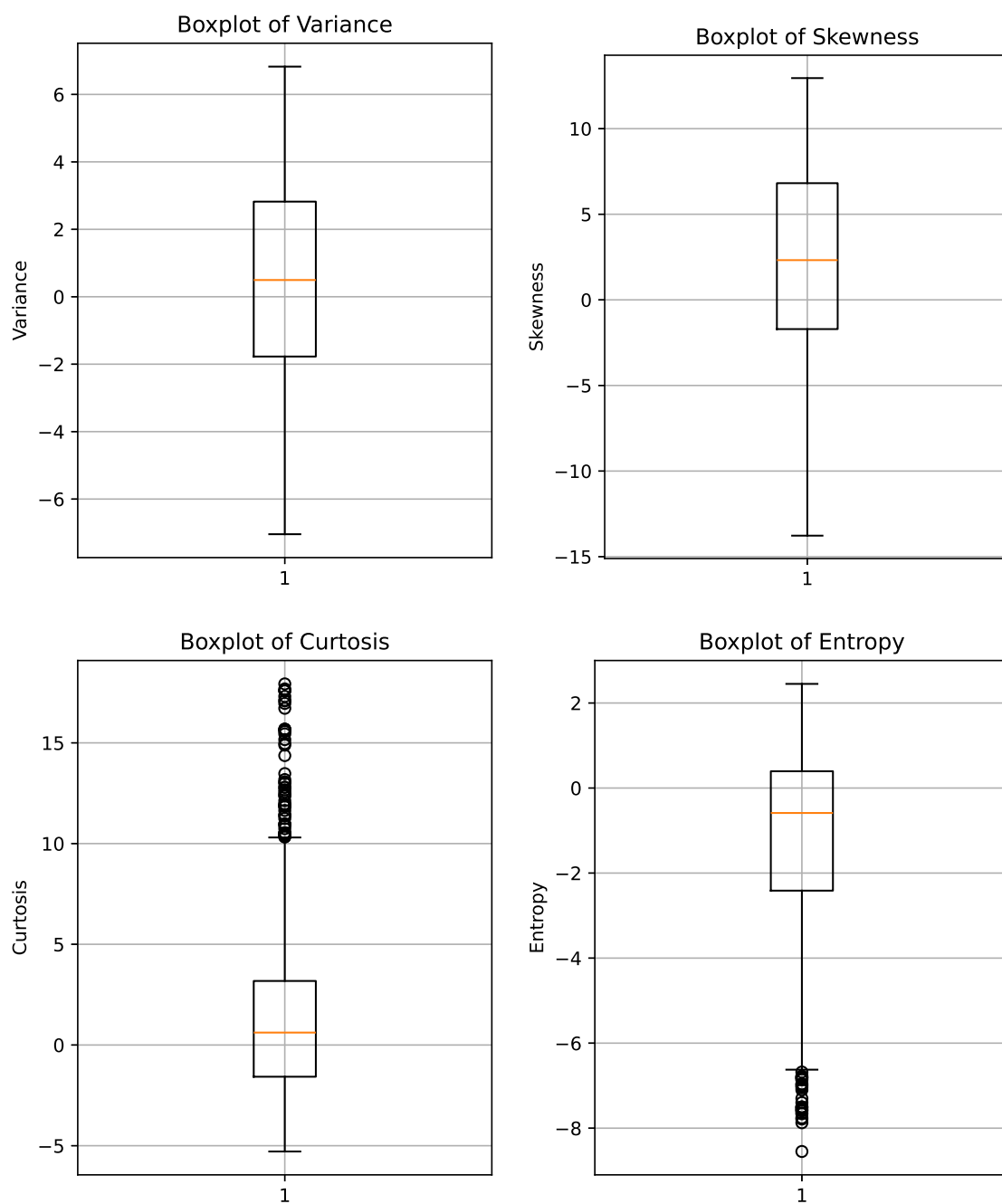


Figure 2: These boxplots show that variance has clear median separation with less overlap between classes, while other features show some extreme outliers.

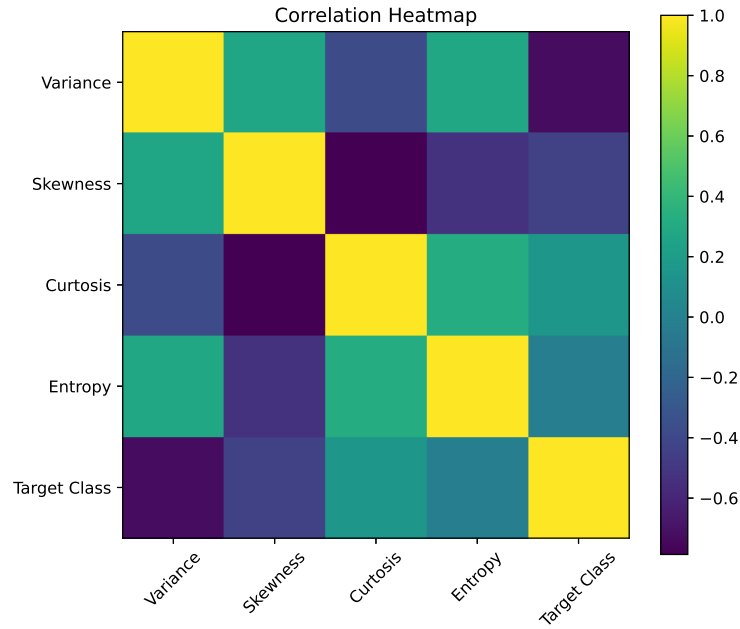


Figure 3: The heatmap shows that skewness and kurtosis have a strong negative correlation with each other, while variance is independent from the others.

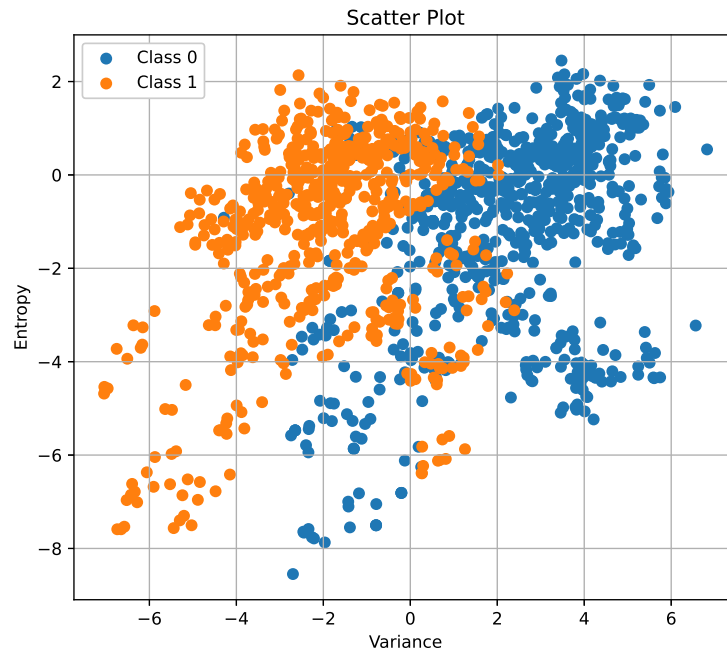


Figure 4: The scatter plot shows that the two classes spread out with clear dividing line between them, which means the data is linearly separable, so the perceptron will be able to classify most of the points correctly.

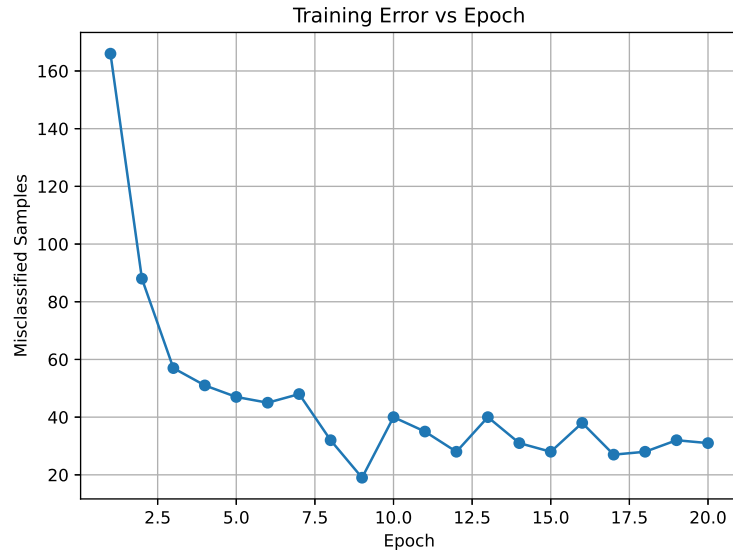


Figure 5: The error graph keeps dropping and decreases to zero by the end, which means the perceptron learned the patterns correctly and converged properly.

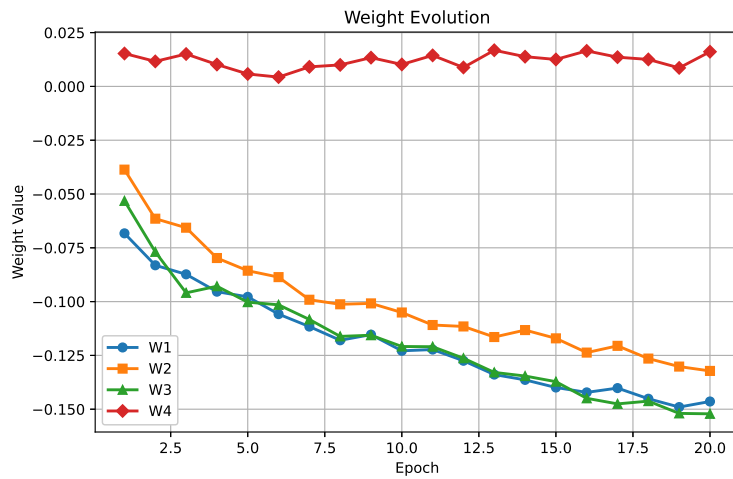


Figure 6: The weights keep changing as the model makes mistakes and finally settles down, which means the perceptron found the best boundary and stopped updating.

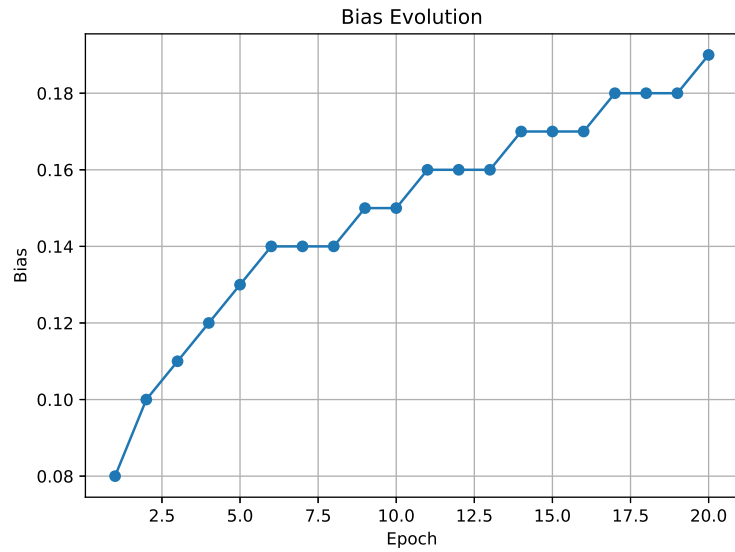


Figure 7: The bias starts at some value and keeps adjusting as the model learns, finally settles down when the perceptron converges.

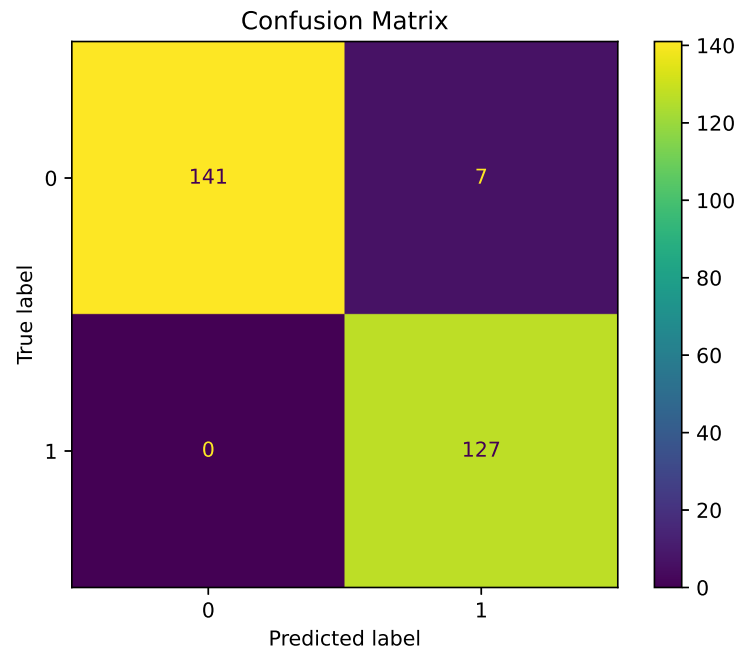


Figure 8: The confusion matrix shows high numbers on diagonal and very few on the off-diagonal values, which means the perceptron classified most notes correctly with only less mistakes.

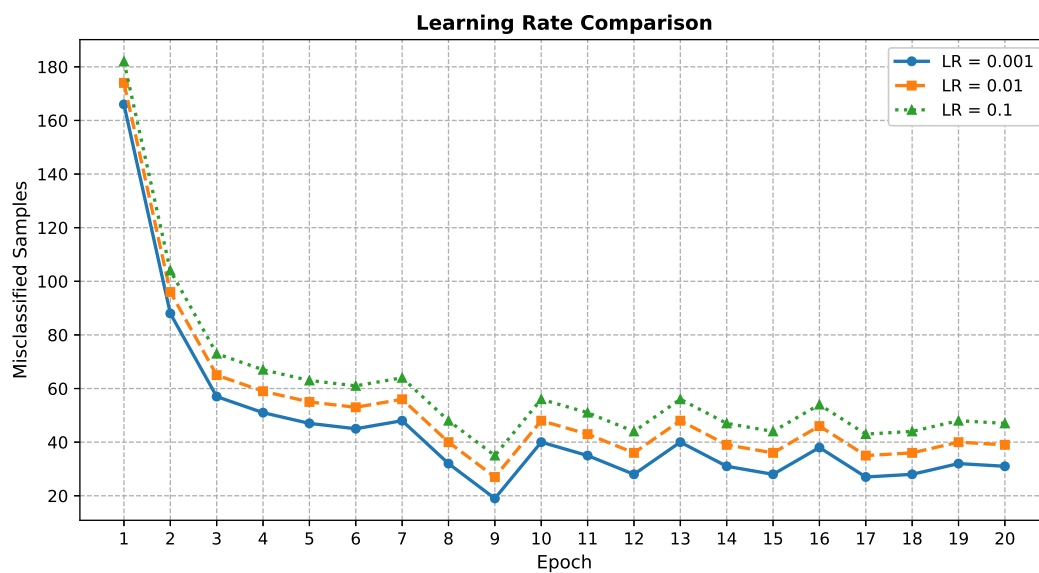


Figure 9: The graph shows that all the three learning rates gives the same error over epochs and changing the learning rate doesn't affect convergence here.

7. Performance Tables

Training Summary

Dataset Size	1372
Train/Test Split	1097 / 275
Learning Rate	0.01
Epochs	20
Final Weights	$[-0.146412, -0.132210, -0.152145, 0.016122]$
Final Bias	0.19
Accuracy	0.974545
Precision	0.947761
Recall	1.000000
F1-score	0.973180

Epoch-wise Learning

Epoch	Errors	Weight 1	Weight 2	Bias
1	166	-0.068274	-0.038669	0.08
2	88	-0.083116	-0.061463	0.10
3	57	-0.087316	-0.065655	0.11

8. Additional Tasks

1. **Compare the Step and Sigmoid activation functions. Discuss why the Step Activation Function is not used in modern deep learning models.**

The Step function gives either 0 or 1 as output and has zero gradient everywhere except at origin where it is not differentiable. Since modern models learn using backpropagation which needs gradients to update weights, Step does not work there. Sigmoid gives a smooth output between 0 and 1 and has non-zero gradients throughout, which allows the model to learn from errors.

2. **Compare the implementation with Scikit-learn's Perceptron.**

The custom perceptron uses a simple loop that runs for a fixed number of epochs with a constant learning rate. It updates one by one. Scikit-learn's version is more advanced since it uses optimized SGD and it has features like early stopping and regularization. So Scikit-learn's version trains much faster and handles larger datasets better.

3. **Repeat the experiment using learning rates 0.001, 0.01 and 0.1.**

From the Learning Rate Comparison plot, 0.01 worked the best since it converged smoothly within 20 epochs. The 0.001 rate was too small so the weights are updated very slowly and need more epochs to converge. The 0.1 rate is too large and causes the errors to jump up and down a lot during training, making the learning unstable.

4. **Explain why a Single Layer Perceptron cannot solve the XOR problem.**

XOR is not linearly separable. The four XOR points are at opposite corners and any straight

line cannot classify at least one of them. Since a single layer perceptron can only draw one linear boundary, it simply can't handle XOR no matter what weights the weights are.

5. Study the effect of feature normalization on model convergence.

The notebook uses MinMax Scaling to bring all features into the range of 0 to 1. Without normalization, features like variance have larger numbers compared to others, so they dominate the weight updates and make the learning process unstable. After normalization, all features contribute equally and the model converges much more smoothly.

9. Discussion

9.1 AND Gate

The AND gate was implemented using a Single Layer Perceptron with the following truth table as the training dataset.

Table 2: Truth Table for AND Gate

x_1	x_2	Output (y)
0	0	0
0	1	0
1	0	0
1	1	1

The perceptron started with zero weights and bias and updated them whenever it made a mistake. It took 8 updates to finally get all predictions right. The final weights and bias define a line that separates the (1,1) point from the rest. Since the AND gate is linearly separable, the perceptron had no trouble finding line and converging successfully.

Weight Updates and Decision Boundaries

Table 3: Perceptron Weight Updates for AND Gate

Update	Input	Target	Prediction	Weights	Bias
1	[0, 0]	0	1	[0.0, 0.0]	-0.1
2	[0, 1]	0	1	[0.1, 0.1]	0.0
3	[0, 0]	0	1	[0.1, 0.1]	-0.1
4	[0, 1]	0	1	[0.1, 0.0]	-0.2
5	[1, 1]	1	0	[0.2, 0.1]	-0.1
6	[0, 1]	0	1	[0.2, 0.0]	-0.2
7	[0, 0]	0	1	[0.1, 0.0]	-0.3
8	[1, 1]	1	0	[0.2, 0.1]	-0.2

Final Learned Parameters

Final Weights: [0.2, 0.1]

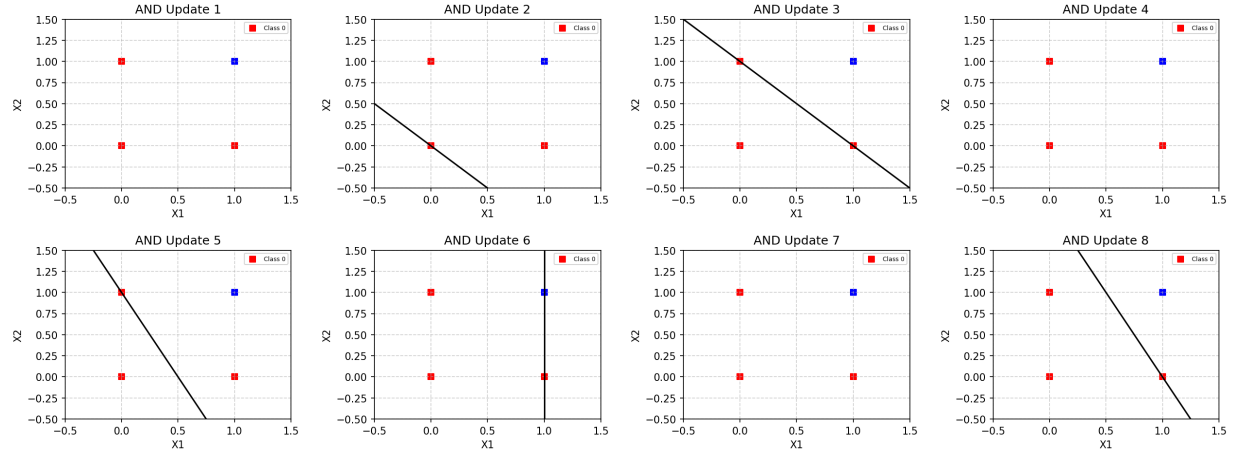


Figure 10: Decision boundaries for the weight updates for AND Gate.

Final Bias: -0.2

Final Predictions

Input	Predicted Output
[0, 0]	0
[0, 1]	0
[1, 0]	0
[1, 1]	1

Final Decision Boundary

Observation:

The perceptron successfully converges for the AND gate because the dataset is linearly separable.

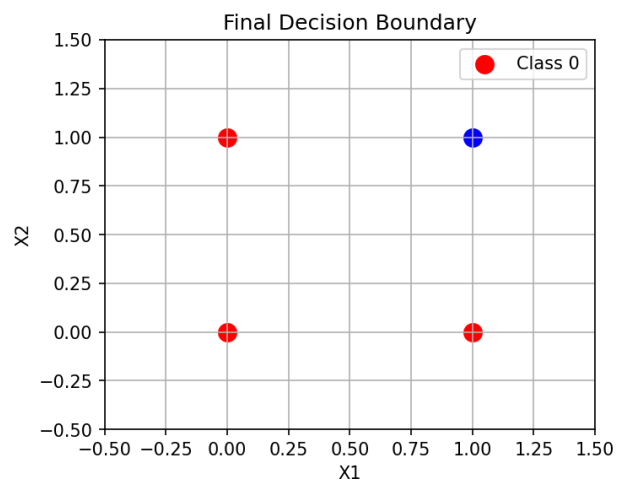


Figure 11: Final Decision Boundary for AND Gate.

9.2 OR Gate

The OR gate was implemented using a Single Layer Perceptron with the following truth table as the training dataset.

Table 4: Truth Table for OR Gate

x_1	x_2	Output (y)
0	0	0
0	1	1
1	0	1
1	1	1

The perceptron started with zeros and updated weights over 5 updates until it converged. The final weights came out as $[0.1, 0.1]$ with a bias -0.1 . The decision boundary separates the single 0 point from the three 1 points. Since OR is linearly separable, the perceptron converged without any issues. **Weight Updates and Decision Boundaries**

Table 5: Perceptron Weight Updates for OR Gate

Update	Input	Target	Prediction	Weights	Bias
1	$[0, 0]$	0	1	$[0.0, 0.0]$	-0.1
2	$[0, 1]$	1	0	$[0.0, 0.1]$	0.0
3	$[0, 0]$	0	1	$[0.0, 0.1]$	-0.1
4	$[1, 0]$	1	0	$[0.1, 0.1]$	0.0
5	$[0, 0]$	0	1	$[0.1, 0.1]$	-0.1

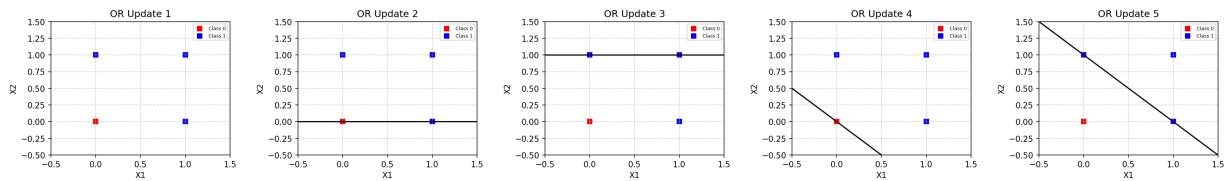


Figure 12: Decision boundaries for the weight updates for OR Gate.

Final Learned Parameters

Final Weights: $[0.1, 0.1]$

Final Bias: -0.1

Final Predictions

Final Decision Boundary

Observation:

The perceptron successfully converges for the OR gate because the dataset is linearly separable.

Input	Predicted Output
[0, 0]	0
[0, 1]	1
[1, 0]	1
[1, 1]	1

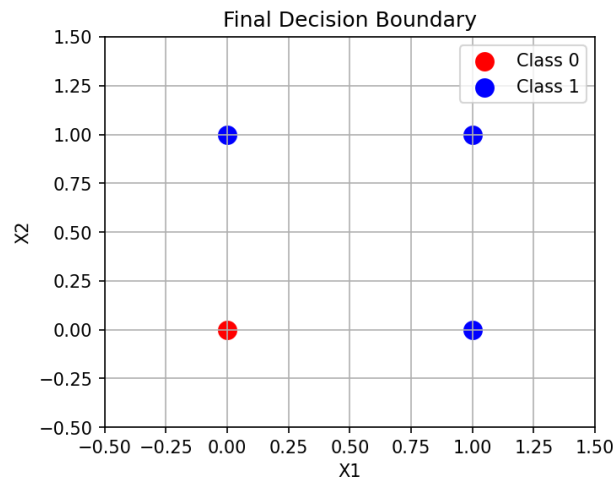


Figure 13: Final Decision Boundary for OR Gate.

9.3 NOT Gate

The NOT gate was implemented using a Single Layer Perceptron with the following truth table as the training dataset.

Table 6: Truth Table for NOT Gate

x_1	Output (y)
0	1
1	0

The perceptron took 2 updates to converge, with the final weight -0.1 and bias 0.0. This creates a threshold that correctly classifies both inputs.

Weight Updates and Decision Boundaries

Table 7: Perceptron Weight Updates for NOT Gate

Update	Input	Target	Prediction	Weights	Bias
1	[0]	1	0	[-0.1]	-0.1
2	[1]	0	1	[-0.1]	0.0

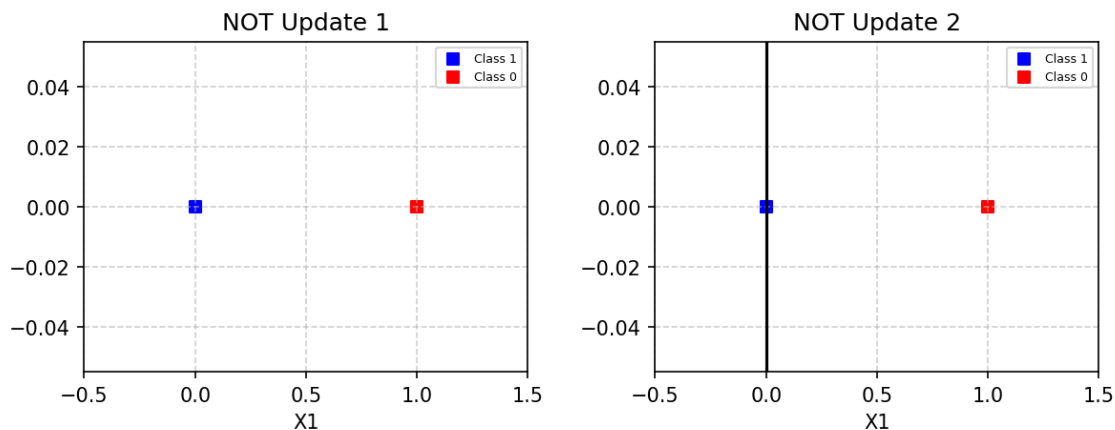


Figure 14: Decision boundaries for the weight updates for NOT Gate.

Final Learned Parameters

Final Weights: [-0.1]

Final Bias: 0.0

Final Predictions

Final Decision Boundary

Observation:

The perceptron successfully converges for the NOT gate because the dataset is linearly separable in 1D space.

Input	Predicted Output
[0]	1
[1]	0

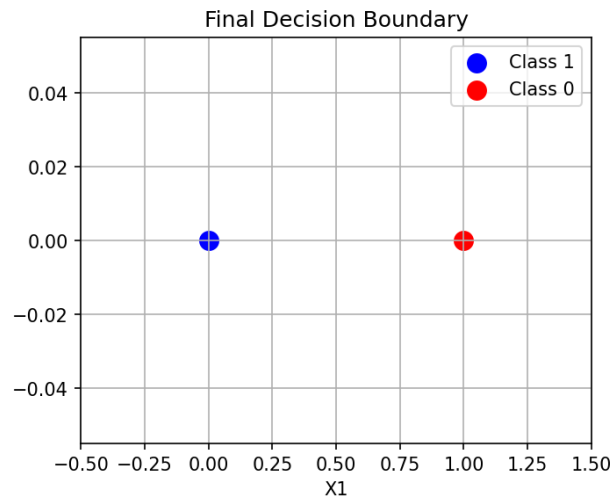


Figure 15: Final Decision Boundary for NOT Gate.

9.4 XOR Gate

The XOR gate was implemented using a Single Layer Perceptron with the following truth table as the training dataset.

Table 8: Truth Table for XOR Gate

x_1	x_2	Output (y)
0	0	0
0	1	1
1	0	1
1	1	0

The perceptron started with zero weights but never converged even after 20 epochs (78 updates). The weights kept changing and final predictions were completely wrong for every input. This proves the fundamental limitation of a Single Layer Perceptron, it can only handle linearly separable data and XOR is not one of them.

Weight Updates and Decision Boundaries

Table 9: Selected Perceptron Weight Updates for XOR Gate

Update	Input	Target	Prediction	Weights	Bias
1	[0, 0]	0	1	[0.0, 0.0]	-0.1
2	[0, 1]	1	0	[0.0, 0.1]	0.0
3	[1, 0]	1	0	[-0.1, 0.0]	-0.1
4	[1, 1]	0	1	[-0.1, 0.1]	0.0
<i>... Updates 5 - 74 ...</i>					
75	[0, 0]	0	1	[-0.1, 0.0]	-0.1
76	[0, 1]	1	0	[-0.1, 0.1]	0.0
77	[1, 0]	1	0	[0.0, 0.1]	0.1
78	[1, 1]	0	1	[-0.1, 0.0]	0.0

Final Learned Parameters

Final Weights: [-0.1, 0.0]

Final Bias: 0.0

Final Predictions

Final Decision Boundary

Observation:

The perceptron fails to converge for the XOR gate because the XOR dataset is not linearly separable, not even a single straight line can separate the two classes, so the weights keep changing during training and algorithm cannot correctly classify all 4 samples, which demonstrates the fundamental limitation of SLP.

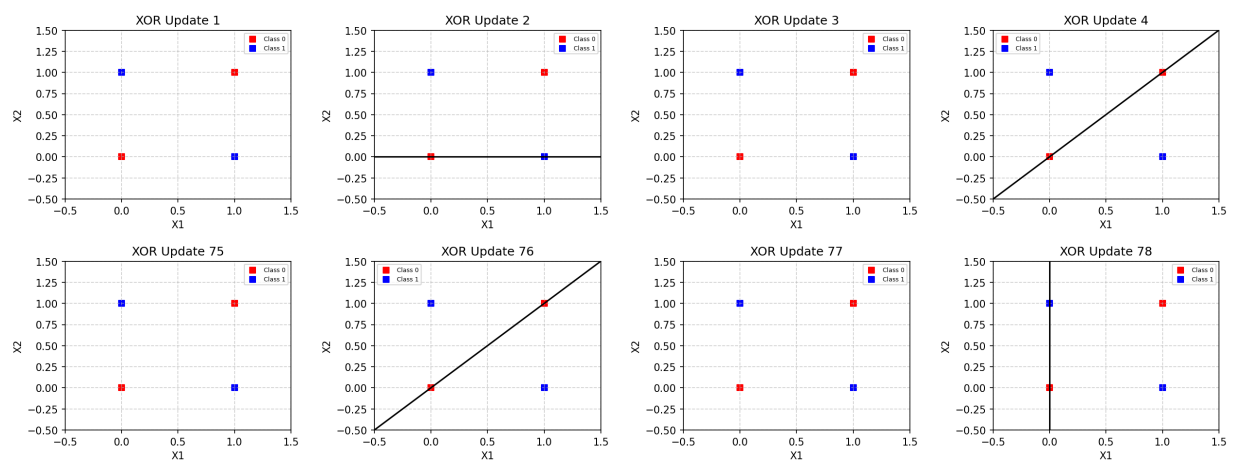


Figure 16: Decision boundaries for the first 4 and last 4 weight updates for XOR Gate.

Input	Predicted Output
[0, 0]	1
[0, 1]	0
[1, 0]	0
[1, 1]	1

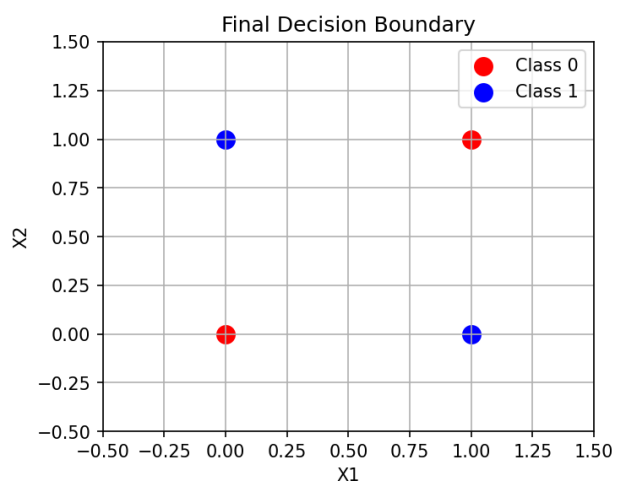


Figure 17: Final Decision Boundary for XOR Gate.

10. References

1. F. Rosenblatt, “The Perceptron,” *Psychological Review*, 1958.
2. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
3. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
4. S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.
5. UCI Machine Learning Repository – Banknote Authentication Dataset.
6. Scikit-learn Documentation: Perceptron.

11. Source Code

- **Colab Notebook:** <https://colab.research.google.com/drive/1FkWNyt7JhQYDpUTc7-uIIInVzhY7qL8a-?usp=sharing>
- **GitHub Repository:** https://github.com/Saraswathi-Nalamothu/deep-learning_experiment-1